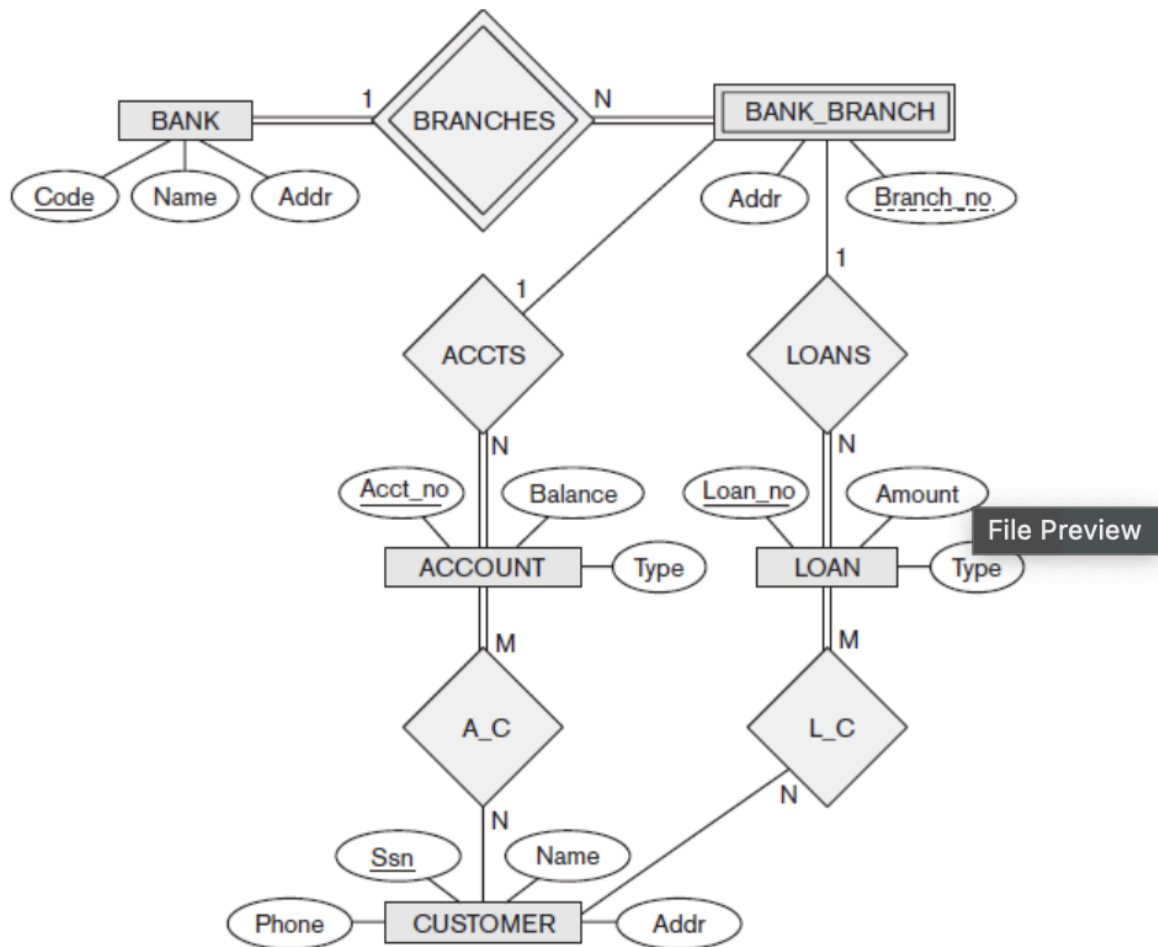




1.

a.

List the strong (nonweak) entity types in the ER diagram.

- Bank
- Account
- Loan
- Customer

b.

Is there a weak entity type? If so, give its name, partial key, and identifying Relationship.

Yes, The weak entity type is **BANK_BRANCH** with partial key Branch_no and identifying relationship **BRANCHES** between **BANK** and **BANK_BRANCH**

c.

What constraints do the partial key and the identifying relationship of the weak entity type specify in this diagram?

The partial key and identifying relationship specify that a BANK_BRANCH cannot exist without a BANK. Branch_no is unique only within a single BANK, not across all banks. The identifying relationship BRANCHES enforces existence dependency, meaning every branch must be related to exactly one bank. The full key for BANK_BRANCH is the combination of Bank.Code and Branch_no.

d.

List the names of all relationship types, and specify the (min, max) constraint on each

participation of an entity type in a relationship type. Justify your choices.

- BRANCHES (BANK–BANK_BRANCH): each BANK can have 1..N branches, each BANK_BRANCH must belong to exactly 1 bank.
- ACCTS (BANK_BRANCH–ACCOUNT): each branch can have 0..N accounts, each account must belong to exactly 1 branch.
- LOANS (BANK_BRANCH–LOAN): each branch can have 0..N loans, each loan must belong to exactly 1 branch.
- A_C (ACCOUNT–CUSTOMER): each account must have 1..M customers, each customer can have 0..N accounts.
- L_C (LOAN–CUSTOMER): each loan must have 1..M customers, each customer can have 0..N loans.

e.

List concisely the user requirements that led to this ER schema design.

1. A bank can have multiple branches.
2. Each branch is uniquely identified within its bank by a branch number.
3. Each branch can manage multiple accounts.
4. Each branch can issue multiple loans.
5. Each account is linked to exactly one branch.
6. Each loan is linked to exactly one branch.
7. A customer can have multiple accounts, and an account can be jointly owned by multiple customers
8. A customer can have multiple loans, and a loan can be taken jointly by multiple customers.
9. Customer information such as SSN, name, phone, and address must be stored.
10. Each account stores its number, balance, and type.
11. Each loan stores its number, amount, and type.
12. Each bank stores its code, name, and address.

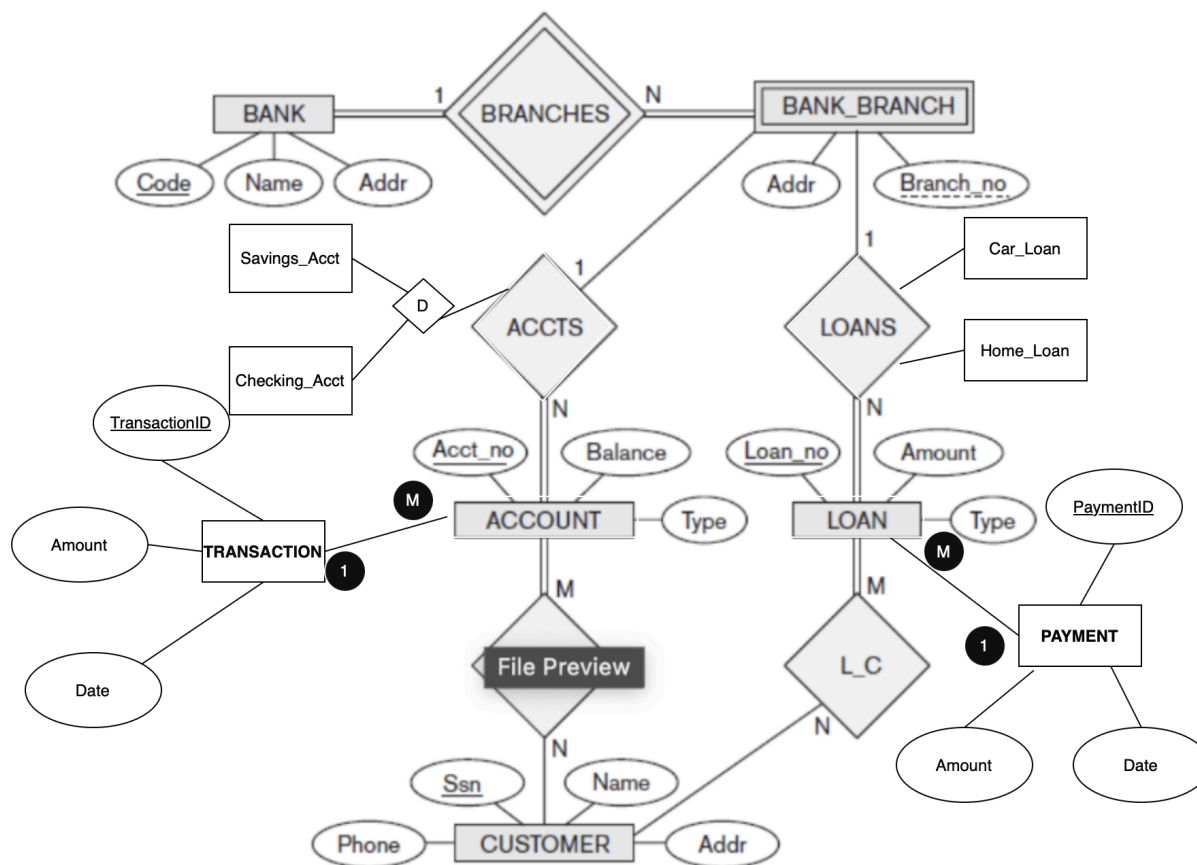
f.

Suppose that every customer must have at least one account but is restricted to at most two loans at a time, and that a bank branch cannot have more than 1,000 loans. How does this show up on the (min, max) constraints?

1. For CUSTOMER participation in the A_C (Account–Customer) relationship: (1, N).
This means every customer must be linked to at least one account.
2. For CUSTOMER participation in the L_C (Loan–Customer) relationship: (0, 2).
This means a customer can have 0, 1, or 2 loans.
3. For BANK_BRANCH participation in the LOANS relationship: (0, 1000).
This means a branch can issue 0 up to 1,000 loans.

Entity 1	Entity 2	Cardinality Ratio	Assumption
Student	SocialSecurity Card	1:1	Each student has one SSN, and each SSN belongs to one student.
Student	Teacher	M:N	A student can have many teachers, and a teacher can teach many students.
ClassRoom	Wall	1:M	A classroom has many walls, but each wall belongs to one classroom.
Country	CurrentPresident	1:1	Each country has exactly one current president.
Course	TextBook	M:N	A course can require many textbooks, and a textbook can be used in courses.

Item (in an order)	Order	M:1	Each item belongs to one order, but an order can contain many items.
Student	Class	M:N	A student can enroll in many classes, and a class can have many students.
Class	Instructor	M:1	Each class is taught by one instructor, but an instructor may teach many.
Instructor	Office	1:1	Each instructor has one office, and each office belongs to one instructor.
E-bay auction item	E-bay bid	1:M	An auction item can receive many bids, but each bid belongs to one item.



ACCOUNTS: The ACCOUNT entity is specialized into SAVINGS_Acct and CHECKING_Acct. This is a disjoint specialization, meaning an account can only be one type. It's also total, meaning every account must be either a savings or a checking account.

LOANS: The LOAN entity is specialized into Car_Loan and Home_Loan. This is a disjoint and total specialization, meaning a loan can only be a car loan or a home loan.

TRANSACTION: This new entity tracks activities for an account. It has attributes for TransactionID (the primary key), Amount, and Date. It is linked to the ACCOUNT entity with a 1-to-N relationship, where one account can have multiple transactions. The relationship is named ACCT_TRANSACTIONS.

PAYMENT: This new entity tracks payments for a loan. It has attributes for PaymentID (the primary key), Amount, and Date. It is linked to the LOAN entity with a 1-to-N relationship, where one loan can have multiple payments. The relationship is named LOAN_PAYMENTS.

Consider the following EER diagram that describes the computer systems at a company. Provide your own attributes and key for each entity type. Supply max cardinality constraints justifying your choice. Write a complete narrative description of what this EER diagram represents.

Attributes and Keys

Computer: ComputerID (Primary Key), SerialNumber, PurchaseDate.

Laptop: ScreenSize, BatteryLife.

Desktop: TowerSize, PowerSupply.

Software: SoftwareID (Primary Key), Name, Version, LicenseKey.

Operating System: OSVersion, BuildNumber.

Accessory: AccessoryID (Primary Key), Brand, Model.

Keyboard: KeyType, Layout.

Mouse: DPI, ButtonCount.

Monitor: Resolution, ScreenSize.

Component: ComponentID (Primary Key), Brand, Model.

Memory: Capacity, Speed.

Sound Card: AudioChannels, ConnectorType.

Video Card: VRAM, OutputPorts.

Relationship Attributes and Constraints

InstalledOS: Connects Computer and Operating System. It's a one-to-many relationship. A computer must have exactly one OS, while an OS can be installed on many computers.

Installed: Links Computer and Software. This is a many-to-many relationship. A computer can have multiple software programs, and a software program can be on multiple computers.

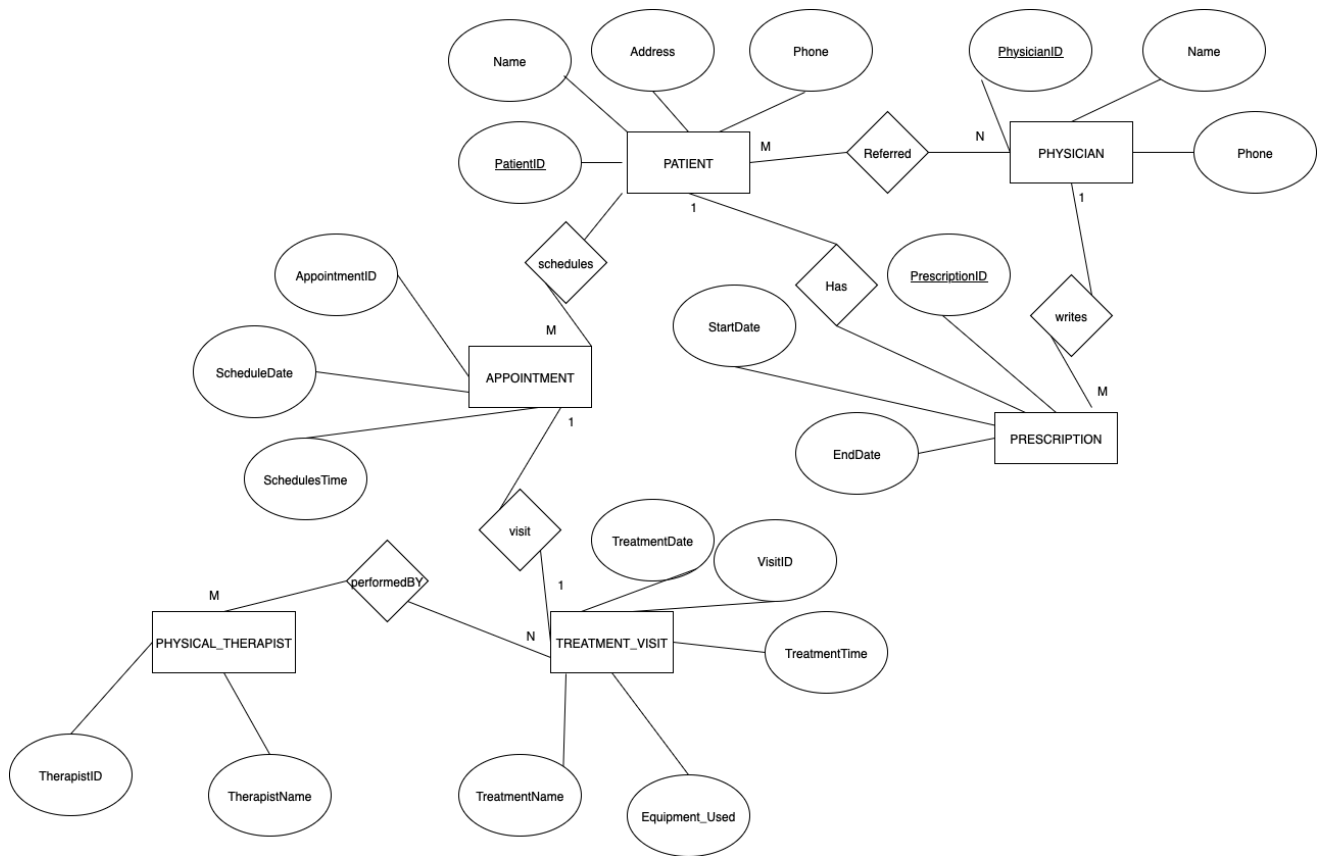
SoldWith: Joins Computer and Accessory. This is a one-to-many relationship from Computer to Accessory. A computer is sold with one or more accessories.

Options: This links Laptop and Desktop to Component. It is a one-to-many relationship. A computer can have many components, but each component belongs to a specific computer.

Supports: Connects Operating System and Software. It is a many-to-many relationship, as an OS can support multiple software applications, and a software application can be supported by multiple OSes.

The maximum cardinalities are based on practical, real-world rules.

- **Computer-Operating System (1,N):** A computer is useless without an OS, so it must have one. However, a single OS type (like "Windows 11") is installed across many computers in a company.
- **Computer-Software (N,N):** A typical computer has many software programs, and any given software product is installed on many computers across the company.
- **Computer-Accessory (1,N):** A computer is always sold with at least one accessory (keyboard, mouse, monitor). These accessory types are then sold with many computers.
- **Computer-Component (0,N):** It's not mandatory for a computer to have extra components, making it an optional relationship. However, if a user decides to add components, they can add more than one.



6.

